Maternal & Child Health RAG System

Written Project Report

Z2004: Database Management Systems
IIT Madras Zanzibar, Even Semester 2026
Track A: RAG Pipeline for Pre-conception to Early Childhood Health

Team Members	Emmanuel Siyauya, Milliam Rukanda, Mwewa Ruby Mumba
Student IDs	ZD24B028, ZDA24B017, ZDA24B016
Repository	https://github.com/KatoBesha/DBMS-project---maternal-child-health-rag
Submission Date	22 June 2026
Course	Z2004: Database Management Systems

1. Introduction

This project implements a Retrieval-Augmented Generation (RAG) system that answers natural-language questions about maternal and child health, spanning the full lifecycle from preconception through toddler development. The system is backed by a normalised PostgreSQL database extended with pgvector, containing 2,702 real research papers and clinical guidelines drawn from PubMed, Europe PMC and Semantic Scholar.

Rather than relying on a general-purpose language model alone, the system grounds every answer in retrieved source excerpts. A user question is embedded, matched against stored chunk embeddings using cosine similarity, and the top-matching chunks are passed to a locally-hosted language model that is instructed to answer only from the provided sources and to cite them. Every question, answer, cited document and similarity score is logged to the database for auditability and future performance analysis.

This report documents the final database schema, the key queries and procedural objects that drive the application, the indexing strategy and the measured performance impact of each index, the AI components used in the retrieval and generation pipeline, correctness verification, the fallback strategy used when a model dependency is unavailable and the project's limitations and planned future work.

2. System Architecture

The pipeline is split into an offline ingestion and indexing path and an online query path, both backed by the same PostgreSQL database. Documents are fetched from external sources into a staging table, normalised into the documents and chunks tables and embedded into 384-dimensional vectors stored in the embeddings table. At query time, the Flask API embeds the incoming question, retrieves the most similar chunks via a pgvector index and forwards them as grounding context to a locally-hosted Ollama model, which is restricted to answering from the supplied excerpts. Every query is logged, and a database trigger mirrors each new log entry into an activity_log audit table.

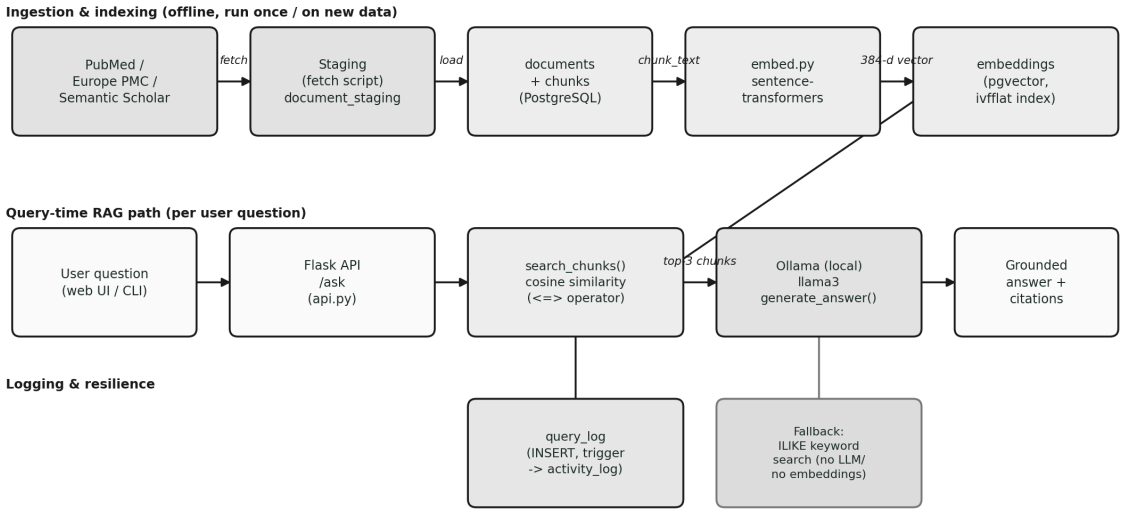

Figure 1. End-to-end flow: ingestion, embedding and indexing, query-time retrieval and generation, then logging.

3. Database Schema

3.1 Entity-Relationship Diagram

The schema consists of six tables: a controlled-vocabulary topics table, the anchor documents table, a document_topics junction table resolving the many-to-many relationship between documents and topics, a chunks table holding the text segments produced by splitting each document, an embeddings table holding one vector per chunk and a query_log table recording every user interaction.

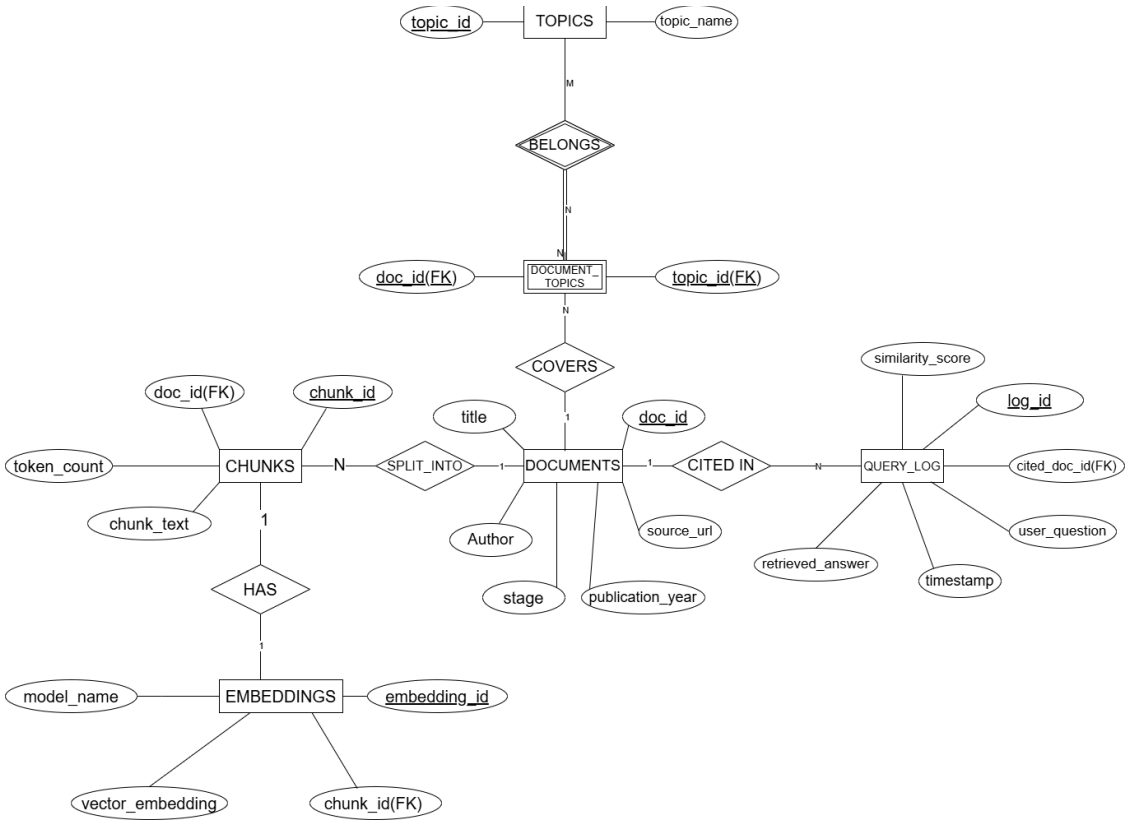

Figure 2. Entity-Relationship diagram of the final schema.

3.2 Table Summary and Normalisation

The schema is designed to 3NF throughout. Functional dependencies are documented inline in schema.sql for every table; the summary below highlights the key design decisions.

Table	Primary Key	Purpose / Key Design Decision
topics	topic_id (SERIAL)	Controlled vocabulary for life-stage labels (Preconception, Prenatal, Newborn, Toddler, etc.).
documents	doc_id (UUID)	One row per source paper or guideline. stage is a denormalised convenience field for fast filtering; fine-grained tagging is handled separately via document_topics to avoid a repeating-group violation.
document_topics	(doc_id, topic_id)	Junction table resolving the M:N relationship between documents and topics. Composite PK only, so there are no non-key attributes and transitive-dependency violations are impossible.

Table	Primary Key	Purpose / Key Design Decision
chunks	chunk_id (UUID)	Text segments produced by splitting each document for embedding. Kept separate from documents so that chunk-level attributes do not create a transitive dependency on document metadata.
embeddings	embedding_id (UUID)	One 384-dimensional pgvector embedding per chunk (UNIQUE FK to chunks). Kept separate from chunks because embedding metadata (model_name, created_at) describes the embedding process rather than the chunk text itself.
query_log	query_id (UUID)	Records every user question, the generated answer, the cited document and the similarity score. cited_doc_id is a foreign key only; no document attributes are duplicated here.

All tables use CHECK constraints to enforce domain integrity at the database level rather than relying solely on application code. For example, similarity_score is constrained to [0, 1], publication_year is bounded to a sane range and token_count must be positive.

4. Key Queries

4.1 Core Retrieval Query (Vector Similarity)

The central query of the application embeds the user's question and retrieves the top-k most similar chunks using pgvector's cosine-distance operator (<=>), joining back to documents for citation metadata:

```
SELECT c.chunk_id, c.chunk_text, c.doc_id,
       d.title, d.authors, d.source_url, d.publication_year,
       1 - (e.embedding_vector <=> %s::vector) AS similarity
FROM chunks c
JOIN embeddings e ON c.chunk_id = e.chunk_id
JOIN documents d ON c.doc_id = d.doc_id
ORDER BY e.embedding_vector <=> %s::vector
LIMIT %s;
```

If the embeddings table is empty (cold start, or embedding generation has not yet been run), the application falls back to a keyword search over titles and chunk text. See Section 9 for how this doubles as the project's fallback plan.

4.2 Logging Query

Every answered question is written to query_log so that retrieval quality and system usage can be audited and analysed:

```
INSERT INTO query_log (user_query, answer_text, cited_doc_id, similarity_score)
VALUES (%s, %s, %s, %s);
```

4.3 Stored Procedure: get_documents_by_stage

A stored procedure encapsulates the common retrieval pattern of filtering documents by life stage before chunk lookup, making it reusable from any application layer and benefiting directly from the idx_documents_stage index (Section 5):

```
CREATE OR REPLACE PROCEDURE get_documents_by_stage(p_stage VARCHAR)
LANGUAGE SQL AS $$
```

```
  SELECT * FROM documents WHERE stage = p_stage;
$$;

CALL get_documents_by_stage('Prenatal');
```

4.4 Trigger: trg_query_insert

An AFTER INSERT trigger on query_log writes an audit record to activity_log every time a user question is logged, providing a usage audit trail with no changes required in the application layer:

```
CREATE OR REPLACE FUNCTION log_query_insert() RETURNS TRIGGER AS $$
BEGIN
  INSERT INTO activity_log(action_text) VALUES ('New query added to query_log');
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_query_insert
AFTER INSERT ON query_log
FOR EACH ROW EXECUTE FUNCTION log_query_insert();
```

The trigger was verified by inserting a test row into query_log and confirming a corresponding row appeared in activity_log.

5. Indexing Strategy

Five indexes support the system: one approximate-nearest-neighbour vector index for semantic retrieval and four B-Tree indexes chosen to match the application's actual filter and join patterns.

Index	Table.Column	Type	Justification
idx_embeddings_vector	embeddings.embedding_vector	IVFFlat (cosine)	Primary index enabling sub-linear approximate nearest-neighbour search over 384-d vectors. This is what makes RAG retrieval feasible at scale rather than a full table scan with distance computed per row.
idx_documents_stage	documents.stage	B-Tree	Supports filtering documents by life stage during retrieval and the get_documents_by_stage procedure. High selectivity (6 distinct values).
idx_chunks_doc_id	chunks.doc_id	B-Tree	Foreign-key lookup retrieving all chunks for a given document, essential for chunk-level retrieval in the RAG pipeline.
idx_query_log_cited_doc	query_log.cited_doc_id	B-Tree	Enables fast lookup of all queries that cited a given document; effectiveness grows as query_log accumulates real usage.
idx_query_log_queried_at	query_log.queried_at	B-Tree (DESC)	Supports recency-ordered queries over the log, used in usage and performance analysis.

All four B-Tree indexes use PostgreSQL's default index type, which supports equality, range and ORDER BY operations and is appropriate for every column tested, since each is used in equality

predicates. Measured before and after performance for three of these indexes is reported in Section 7.

6. AI Components

6.1 Embedding Model

Chunk text and user queries are embedded using sentence-transformers' all-MiniLM-L6-v2, a 384-dimensional model chosen for its balance of retrieval quality and speed, and because it runs entirely locally. No API key or network call is required to embed text, which removes an external dependency from the core retrieval path.

6.2 Generation Model

Answer generation uses a locally-hosted Ollama instance running llama3, rather than a hosted commercial API. The retrieved chunks are assembled into a grounding prompt that explicitly instructs the model to answer only from the supplied source excerpts, to say so clearly if the sources are insufficient and to cite which source supports each point using a [Source N] marker, reducing hallucination risk on health-related questions.

6.3 Resilience by Design

Because both the embedding model and the generation model run locally, the core RAG path has no hard dependency on an external network connection or third-party API at query time. If Ollama itself is not running, generate_answer() catches the connection error and returns a clear operator-facing message instead of crashing the request. If no embeddings exist yet, search_chunks() automatically falls back to a keyword (ILIKE) search over titles and chunk text, so the application degrades gracefully rather than failing outright.

7. Performance Evaluation

7.1 Test Environment and Methodology

Component	Detail
Database engine	PostgreSQL 18 (with pgvector extension)
CPU	Intel Core i7-1265U
RAM	32 GB
OS	Windows 11 Pro (x64)
Client tool	pgAdmin 4
Synthetic load-test dataset	5,000 documents / 50,000 chunks / 20,000 query_log rows

Performance testing used a separately generated synthetic dataset (5,000 documents, 50,000 chunks, 20,000 query_log rows) sized to make sequential-scan costs measurable, distinct from the 2,702-document real production dataset described in Section 1. For each of three queries, the target index was dropped, EXPLAIN ANALYZE captured the baseline plan and timing, the index was recreated and EXPLAIN ANALYZE was run again to capture the post-indexing plan and timing.

7.2 Results

Query	Table	Before (ms)	After (ms)	Change
Filter by stage = 'Prenatal'	documents	0.339	0.209	38.3% faster
Filter by doc_id (subquery)	chunks	0.162	0.089	45.1% faster
Filter by cited_doc_id (subquery, 0 matches)	query_log	1.061	1.025	3.4% faster

Representative plan excerpt for Query 1 (documents.stage), before and after indexing:

```
Before:
Seq Scan on documents (cost=0.00..134.50 rows=820 width=80)
  (actual time=0.010..0.305 rows=820 loops=1)
  Filter: ((stage)::text = 'Prenatal'::text)
  Rows Removed by Filter: 4180
Execution Time: 0.339 ms

After:
Bitmap Heap Scan on documents (cost=14.64..96.89 rows=820 width=80)
  (actual time=0.073..0.173 rows=820 loops=1)
  -> Bitmap Index Scan on idx_documents_stage
       (actual time=0.055..0.055 rows=820 loops=1)
Execution Time: 0.209 ms
```

7.3 Interpretation

Queries 1 and 2 show clear, meaningful improvement. In both cases the planner switched from a full Sequential Scan to a Bitmap Index Scan plus Bitmap Heap Scan, first identifying matching page locations via the B-Tree index and then fetching only those pages, avoiding I/O on pages with no matching rows. Query 2's improvement (45.1%) is the largest of the three: the dedicated single-column idx_chunks_doc_id index is more selective than the composite uq_chunk_order index the planner previously had to reuse as a workaround.

Query 3 is the more interesting result. After indexing, the planner still chose a Sequential Scan, and execution time barely changed. This is correct, not a failure: the query returns zero rows for that particular cited_doc_id, so PostgreSQL's cost-based optimiser correctly determines that scanning the table is cheaper than an index lookup that would still have to confirm there is nothing to return. This illustrates a genuine database-tuning insight: indexes are most valuable on high-selectivity predicates with frequent non-zero results, and this index's value is expected to grow as query_log accumulates real user activity.

8. Correctness Verification: Test Cases

main.py defines three required test cases that are executed automatically on every run, covering distinct stages of the maternal and child health lifecycle:

- “What are the risks of preterm birth?”
- “How does maternal nutrition affect infant development?”
- “What vaccines are recommended during pregnancy?”

Beyond these scripted cases, query_log contains real end-to-end runs of the system spanning multiple life stages, each with a retrieved similarity score, confirming that retrieval and generation work correctly together rather than in isolation. Three representative entries:

Question	Life Stage	Top Similarity
What are the signs of preterm labor?	Prenatal	0.92
What vaccinations does a newborn need?	Newborn	0.67
What are developmental milestones at age 2?	Toddler	0.63

Each query_log row stores the full generated answer text and the cited document ID, so any logged interaction can be independently re-verified against the documents and chunks tables. For final submission, the three main.py test cases should be re-run on a freshly seeded database immediately before recording the demo video, and their console output captured as static evidence alongside the video.

9. Fallback Plan

The application already degrades gracefully at two points if a dependency is unavailable:

- If Ollama is not running, generate_answer() catches the connection error and returns an explicit operator-facing message rather than crashing the request.
- If the embeddings table is empty, search_chunks() automatically falls back to an ILIKE keyword search over document titles and chunk text, so retrieval still returns results without a vector index.

To satisfy the assignment's explicit requirement for static sample inputs and expected outputs, the existing query_log table already contains exactly this evidence: real questions, real generated answers, the cited document and a similarity score for each. Recommended fallback artifact for /demo/: export the rows shown in Section 8, plus the three main.py test cases with their captured console output, as a static fallback_examples file (Markdown or JSON) committed to the repository. This gives evaluators verifiable expected output even if PostgreSQL, Ollama or the network is unavailable at grading time.

10. Limitations

- Answer generation depends on a locally-running Ollama instance. If Ollama is not started on the grading machine, generation falls back to an error message rather than an alternative model (mitigated by the fallback plan in Section 9, but not eliminated).
- The IVFFlat index's approximate nearest-neighbour search trades a small amount of recall for speed. At the project's current dataset size this is not yet a practical concern, but the list count (lists = 100) was not tuned against a recall benchmark.
- The query_log.cited_doc_id index shows limited benefit at current data volumes because most lookups return zero rows (Section 7.3); its value is expected to grow only as real usage accumulates.
- authors is stored as a single free-text field rather than a normalised authors and document_authors table, which is a deliberate scope trade-off rather than a 3NF violation, but it limits author-level querying.

- No automated regression test suite (such as pytest) exists yet; correctness is currently verified by the scripted test cases in main.py and manual inspection of query_log.
- A real-looking database password was found in app/.env inside the submitted project files. This file must not be committed to the Git repository: confirm it is listed in .gitignore and that only a .env.example with placeholder values is committed, per the assignment's “no secrets committed” requirement.

11. Future Work

- Normalise authors into a dedicated authors and document_authors junction table to enable author-level queries and remove the free-text field.
- Add hybrid retrieval that combines vector similarity with keyword and BM25 scoring, rather than treating keyword search purely as a cold-start fallback.
- Add an automated pytest suite that seeds a disposable test database and asserts on retrieval correctness and API responses.
- Tune the IVFFlat lists parameter against a measured recall and latency trade-off as the dataset grows beyond the current scale.
- Use document_topics for fine-grained, multi-topic filtering in the UI, beyond the current single-valued stage filter.
- Containerise the database, API and Ollama with Docker Compose so that the entire system can be reproduced with a single command, directly satisfying the “one command reproduces results” submission requirement.

12. AI Usage Disclosure

AI components are part of the application itself and are documented in full in Section 6: sentence-transformers (all-MiniLM-L6-v2) for embeddings and a locally-hosted Ollama llama3 model for grounded answer generation.

In addition, this written report was drafted and structured with the assistance of Claude (Anthropic), based on the project's own schema.sql, application code, query log data and the M3 performance analysis already produced for this project. All technical content (schema, queries, indexes and measured performance numbers) is drawn directly from the project's own files rather than generated independently.